

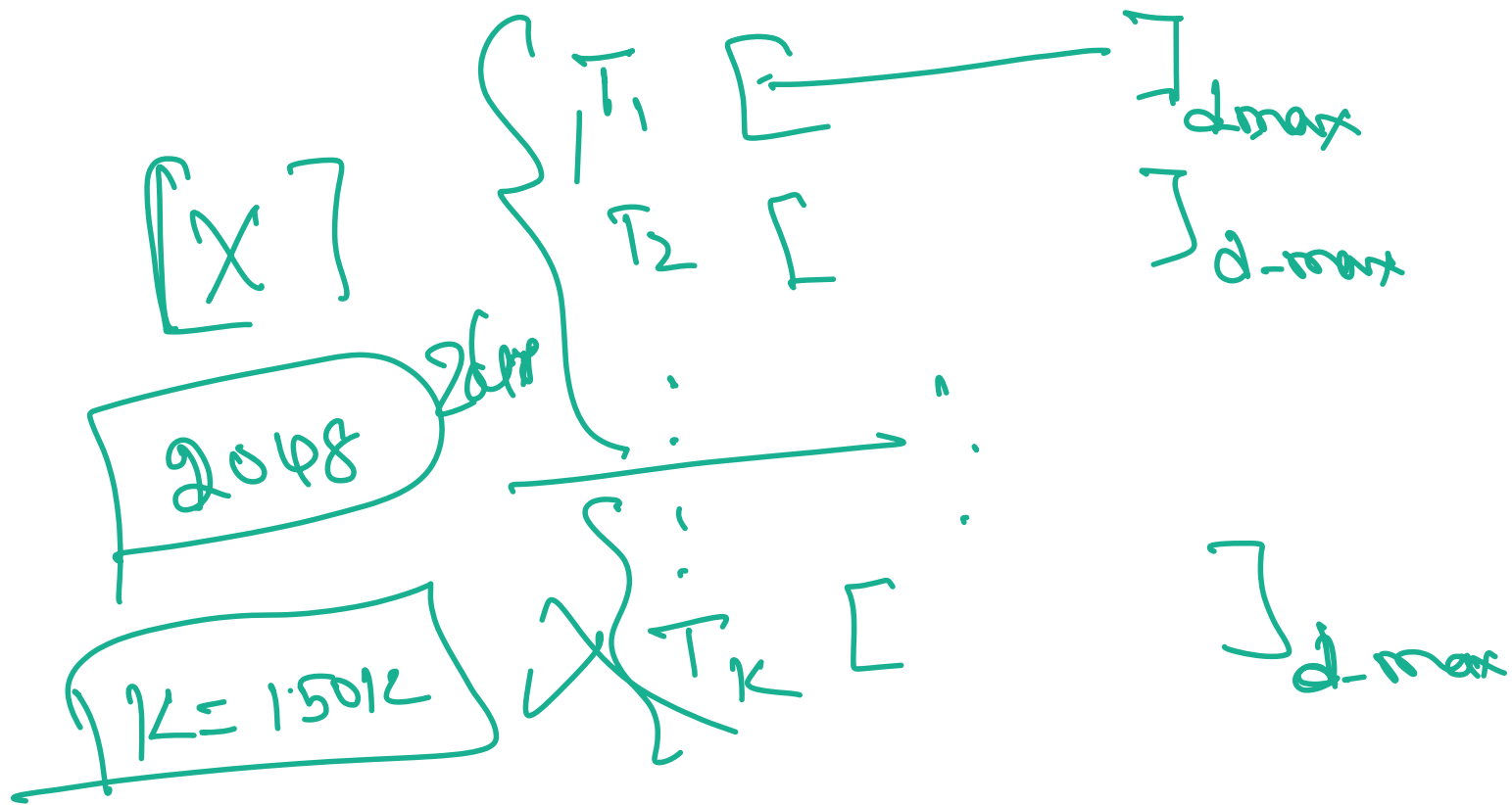
AhamX Bodhi

Where I Becomes Infinite

Max Tokens and Context Length

Module 3.1: Working with LLM APIs

Prof. Sashikumaar Ganesan | IISc Bengaluru | 2026





Session Overview

Prerequisites

- What tokens are and how tokenization works
- Autoregressive generation process
- Basic LLM API structure and parameters
- Transformer attention mechanism (conceptual)

Q. I ... ?
I like ?
I like the ?

What You Will Learn

- Difference between context length and max tokens
- How the context window limits working memory
- Cost and latency implications of token budgets
- Context window sizes across major models
- Strategies for managing long contexts



Two Critical Token Concepts

Context Length vs. Max Tokens — Not the Same Thing

Context Length (context window) is the *maximum total tokens* the model can process in one call — input tokens plus output tokens combined. This is a hard architectural limit set at training time.

Max Tokens (`max_tokens` in the API) is a *developer-specified cap* on how many tokens the model may generate in its response. This is a soft budget parameter, not an architectural limit.

The Relationship Between the Two

- $\text{input_tokens} + \text{max_tokens} \leq \text{context_length}$
- If the prompt is large, less room remains for the output
- A 128K context window with a 100K-token prompt leaves only 28K tokens for the response — even if `max_tokens=60000` is set
- Exceeding the context limit produces a `context_length_exceeded` error or silently truncates the oldest messages



Every token in the context window is processed at every generation step. Longer contexts increase compute cost substantially.



Max Tokens as a Budget Control Parameter

Why Set Max Tokens?

- **Cost control:** Output tokens are billed; capping prevents runaway charges from verbose responses
- **Latency control:** Each output token adds latency; capping bounds worst-case response time
- **UX constraint:** UI components have limited space; long responses may overflow display widgets
- **Safety margin:** Leave buffer tokens between input length and context limit to avoid errors

Max Tokens Pitfalls

- Setting it too low truncates responses mid-sentence, producing incomplete answers
- The model does not know truncation is coming; it cannot gracefully summarise before stopping
- A budget fitting average responses may fail on complex multi-step queries
- Best practice: set 20–30% above expected output length, not at the minimum possible value



Context Window Sizes --- Major Models (2024--2025)

Model	Provider	Context Window	Notes
GPT-4o	OpenAI	128K tokens	Standard frontier model
GPT-4o-mini	OpenAI	128K tokens	Smaller, faster, cheaper
o1 / o3	OpenAI	128K tokens	Extended reasoning models
Claude 3.5 Sonnet	Anthropic	200K tokens	Largest API context window
Claude 3.5 Haiku	Anthropic	200K tokens	Fast, low-latency model
Gemini 1.5 Pro	Google	1M tokens	Video and audio context
Gemini 1.5 Flash	Google	1M tokens	High-throughput variant
Llama 3.1 405B	Meta	128K tokens	Open-weight frontier model
Mistral Large	Mistral	128K tokens	Open-source oriented

Context windows have grown $32\times$ since GPT-3 (4K tokens, 2020).



Context Window as the Model's Working Memory

LLMs Have No Persistent Memory Beyond the Context

A language model has no memory between API calls. Everything it “knows” about the current task — instructions, history, retrieved documents, user preferences — must be present in the context window of each call. Once a conversation exceeds the limit, the oldest tokens must be dropped or compressed.

Practical Token Budget Examples

- A 10-turn chat with 500 tokens per turn uses 5,000 tokens — well within most modern models
- A RAG pipeline with 20 retrieved chunks of 500 tokens each consumes 10,000 input tokens before the user query is added
- A system prompt of 2,000 tokens is repeated and billed on *every* API call — keep system prompts concise
- Long code files, PDFs, and transcripts can each exceed the context windows of smaller or older models



Tokens vs. Words --- Practical Estimation

Rule of Thumb: 1 Token $\approx$ 0.75 Words in English

Token counts vary by language and tokeniser. English text tokenises at approximately 1 token per 0.75 words (roughly 4 characters per token).

Content	Approx. Tokens	Fits in GPT-4o (128K)?
Short email (200 words)	$\sim$ 260	Yes (easily)
Blog post (1,000 words)	$\sim$ 1,300	Yes
Academic paper (8,000 words)	$\sim$ 10,400	Yes
Short novel (80,000 words)	$\sim$ 104,000	Yes (tight)
Long novel (200,000 words)	$\sim$ 260,000	No — exceeds 128K
Python codebase (50K LOC)	$\sim$ 200,000	Needs 200K (Claude)

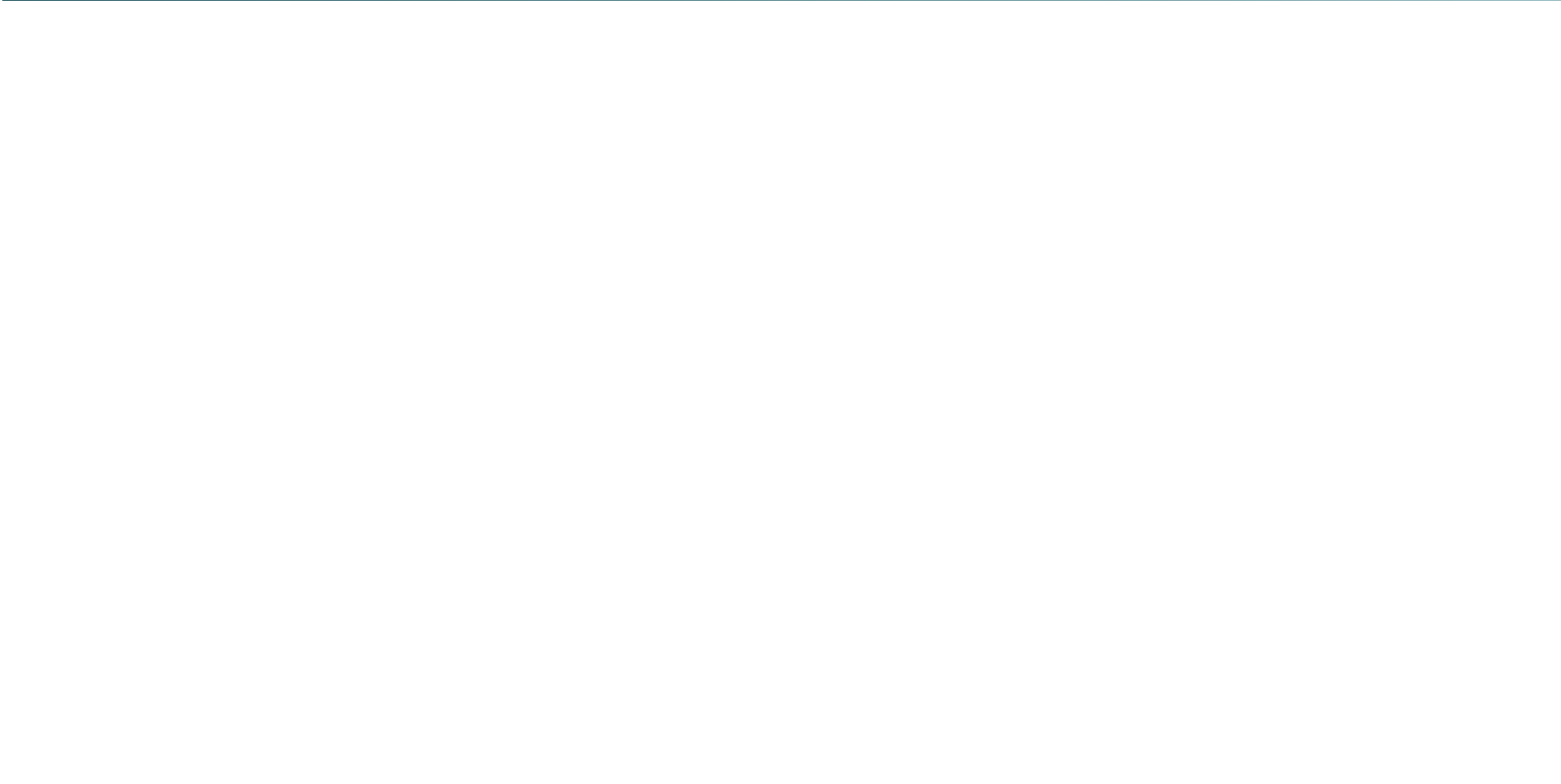
Use the `tiktoken` Python library to count tokens exactly before an API call.



Strategies for Managing Long Contexts

Five Approaches When Context Limits Are a Constraint

- **Chunking and RAG:** Divide long documents into chunks; retrieve only the relevant 3–5 chunks per query using vector search
- **Conversation summarisation:** Periodically compress older turns into a brief summary; discard raw turns while preserving semantics
- **Sliding window:** Keep only the most recent n turns of history; sacrifice early context for recency and bounded cost
- **Prompt caching:** Cache the static system prompt and repeated context prefix (Anthropic and OpenAI both offer this) to avoid re-paying for tokens that do not change between calls
- **Model selection:** Choose the model whose context window fits the task — GPT-4o-mini for short tasks, Claude 3.5 Sonnet for large document analysis





GenAI Application: Long Document Analysis

How Large Contexts Enable New Use Cases

- **Legal due diligence (Harvey AI):** Feeds entire merger contracts (50–100K tokens) into a single Claude call. 200K context allows cross-referencing clauses from different sections without chunking-induced context loss
- **Financial report analysis:** Annual reports of large companies are 50–150 pages. A 200K context window fits the full report for holistic analysis and cross-section reasoning in one call
- **Code review (GitHub Copilot Enterprise):** Entire repositories fit in large-context models for cross-file refactoring and dependency tracing
- **Academic research (Consensus, Elicit):** Multiple papers loaded simultaneously for multi-document synthesis and citation cross-referencing

Large context windows reduce the need for complex RAG orchestration in document analysis.



GenAI Application: Multi-Turn Conversation Management

Token Growth in Chat

- Every user message is re-sent on each API call
- A 30-minute support conversation accumulates 8,000–15,000 tokens
- Most bots implement auto-summarisation: after 10 turns, the first 5 are compressed and dropped
- This keeps context cost bounded regardless of conversation length

LangChain Memory Options

- `ConversationBufferMemory`: Full history; simplest but unbounded cost growth
- `ConversationSummaryMemory`: Auto-summarises older turns; bounded context cost
- `ConversationTokenBufferMemory`: Maintains last N tokens; precise budget control
- `VectorStoreMemory`: Retrieves relevant history per query using embeddings



GenAI Application: Gemini 1.5 Pro and 1M-Token Windows

New Use Cases Enabled by Million-Token Contexts

- **Hour-long video analysis:** A 1-hour video at standard quality encodes to $\sim 750K$ tokens. Gemini 1.5 Pro analyzes the full video in one call, enabling temporal reasoning across the whole recording
- **Entire codebase review:** Large projects with 500K lines of code fit in Gemini's context for architecture analysis without chunking
- **Multi-book research:** Loading 4–5 full-length books simultaneously for cross-reference synthesis and thematic analysis in one prompt
- **Full audio transcripts:** Meeting recordings of 8 hours or more can be analyzed for themes, speakers, and action items in one API call



GenAI Application: Prompt Caching for Cost Reduction

How Prompt Caching Works

- A large static context is cached server-side after the first call; subsequent calls reuse KV states, paying only for new tokens
- Anthropic: up to 90% cost reduction on the cached prefix; cache lasts 5 minutes
- OpenAI: automatic caching for prompts over 1K tokens; 50% discount on cached input tokens
- Break-even reached at 2+ calls with same prefix

Ideal Use Cases

- RAG systems where the same large document is queried multiple times per session
- Multi-user apps where all users share the same large system prompt and tool definitions
- Agent loops that re-query with growing context; only new tokens are billed each turn
- Development and testing: same prompt sent many times with small parameter variations

Session Summary



Key Takeaways

1. **Context length** is the total token budget (input + output) fixed at training time; **max_tokens** is the developer cap on output length per request — two distinct limits
2. The governing formula: $\text{input_tokens} + \text{max_tokens} \leq \text{context_length}$
3. The context window holds *all* information the model can access: system prompt, history, retrieved docs, and user query
4. English text tokenises at ≈ 1 token per 0.75 words; count with `tiktoken` before calling the API
5. Major models in 2025 offer 128K–1M token contexts; choose based on document size and cost requirements
6. Manage growing contexts with RAG chunking, conversation summarisation, sliding window, or provider-level prompt caching

AhamX Bodhi

Where I Becomes Infinite